

Jenkins CI/CD

1. CI/CD

Continuous Integration (CI) is the practice of frequently integrating code changes into a shared repository and automatically building and testing those changes. In this practical, a GitHub push can notify Jenkins, which then runs the pipeline stages. Continuous Delivery/Deployment (CD) extends CI by automating preparation and, depending on the setup, release or deployment of validated software. This practical mainly demonstrates the CI portion:

GitHub → Webhook → Jenkins → Build → Test.

2. Jenkins Architecture

Jenkins commonly uses a Controller-Agent architecture. The Jenkins Controller manages jobs, pipeline orchestration, credentials, scheduling, and communication with agents. A Jenkins Agent performs the actual workload assigned by the controller. In this practical, the Jenkins Controller runs in a Docker container on the Linux Mint host, while a Jenkins Agent runs inside an Ubuntu virtual machine. The agent executes the pipeline workload. Component Role in this practical
GitHub Stores source code and Jenkinsfile; sends webhook events. Cloudflare Quick Tunnel Provides a temporary public HTTPS endpoint for the local Jenkins controller. Jenkins Controller Runs in Docker on the Linux Mint host and manages the pipeline. Ubuntu VM Agent Executes Checkout, Build, and Test steps.

3. Freestyle Jobs

A Jenkins Freestyle Job is a traditional Jenkins job configured primarily through the Jenkins web interface. It can define source-code management, build triggers, build steps, and post-build actions. Freestyle jobs are useful for simple tasks, but complex configurations can become harder to version and maintain.

4. Declarative Pipelines

A Declarative Pipeline is a structured way to define Jenkins automation as code. It uses a clear syntax based around a pipeline, agent, stages, and steps. This makes the CI process easier to read, version, review, and maintain. The practical pipeline is organized as: Checkout → Build → Test.

5. Jenkinsfile

A Jenkinsfile is a text file containing the Jenkins Pipeline definition. It is normally stored in the source code repository so that pipeline configuration is version-controlled together with the application.

6. Pipeline Stages

Stage Purpose Checkout Retrieves the source code from the configured Git repository.
Build Runs the commands needed to build or prepare the application. Test Runs tests or basic validation and reports whether they pass or fail.

7. GitHub Webhooks

A GitHub Webhook sends an HTTP request to a configured service when a selected GitHub event occurs. For this practical, the push event is used. GitHub sends the event to Jenkins, which can then trigger the configured Pipeline. Because the Jenkins controller is running locally, a Cloudflare Quick Tunnel was used to expose a temporary public HTTPS endpoint. The webhook endpoint follows the Jenkins GitHub webhook path: `https://<public-jenkins-url>/github-webhook/`. The webhook uses application/json and SSL verification remains enabled. The Quick Tunnel is suitable for testing and practical work, not as a permanent production endpoint.

8. Jenkins Credentials

Jenkins Credentials provide secure storage for authentication information such as GitHub tokens, SSH keys, usernames/passwords, and secret text. Credentials should be referenced by their Jenkins credential ID rather than exposing secrets directly in a Jenkinsfile or source repository.

9. Practical Workflow

Git push → GitHub → Webhook → Cloudflare HTTPS tunnel → Jenkins Controller → Jenkinsfile
→ Checkout → Build → Test → Jenkins Agent

A successful pipeline is verified through the Jenkins build page and Console Output, where the stages and final result such as Finished: SUCCESS can be inspected.